

# RAG-Based YouTube Chatbot

## Project Report (Updated Version)

### Abstract

This project implements a Retrieval-Augmented Generation (RAG) chatbot that answers questions about any YouTube video by grounding its responses in the video's transcript. The transcript is split into chunks, embedded locally with a sentence-transformer model, indexed with FAISS, and the most relevant chunks are passed to Google Gemini to generate an answer. The system is exposed through a Streamlit chat interface that shows the transcript excerpts behind every answer, supports follow-up questions, and can summarize the whole video.

This updated version brings the original project up to date with current library versions, fixes several defects that prevented it from running (missing dependencies, an outdated transcript API, a broken subtitle parser, a mismatch between the documented FAISS store and the implemented Chroma store, and a retired Gemini model), and adds chat memory, whole-video summarization, automatic model fallback and clearer error handling. All results in this report were measured on the updated system.

# 1. Introduction

A large share of today's educational and technical content is published as video. Large Language Models (LLMs) cannot watch a video, and when asked about one they either refuse or answer from their training data, which may be outdated or simply wrong. This project closes that gap by using the video's transcript as an external knowledge base.

The chatbot fetches the transcript of a YouTube video, turns it into a searchable vector index, and for every question retrieves the most relevant passages and asks the LLM to answer *only* from them. This is the Retrieval-Augmented Generation pattern: the answer is grounded in real source text, and the user can see exactly which passages were used.

## 2. Objectives

- Design and implement a complete RAG pipeline for YouTube videos.
- Build a transcript-based knowledge base with local embeddings and a FAISS vector index.
- Generate grounded answers with Google Gemini, and say so when the video does not contain the answer.
- Provide an interactive Streamlit chat interface with follow-up questions and whole-video summaries.
- Make the system robust: clear errors, transcript fallbacks and automatic LLM model fallback.
- Validate the system with functional tests, a question-answering check and measured performance.

## 3. Background

### 3.1 Retrieval-Augmented Generation (RAG)

RAG (Lewis et al., 2020) adds a retrieval step before generation: relevant passages are fetched from an external store and inserted into the prompt. This reduces hallucinations, keeps answers up to date without retraining the model, and makes it possible to show the evidence behind an answer.

### 3.2 Sentence embeddings

**all-MiniLM-L6-v2** from the Sentence-Transformers library maps each text chunk to a 384-dimensional vector so that passages with similar meaning lie close together. It is small (about 90 MB), runs locally on CPU or Apple GPU, and needs no API key.

### 3.3 FAISS

FAISS (Facebook AI Similarity Search) is a library for fast nearest-neighbour search over vectors. The project uses an in-memory FAISS index built through LangChain; embeddings are normalized, so L2 distance ranks chunks the same way as cosine similarity.

### 3.4 LangChain and Google Gemini

LangChain provides the building blocks: text splitter, embedding wrapper, vector store, prompt templates and the Gemini chat model wrapper, including its fallback mechanism. Google Gemini generates the answers; the free tier of the Gemini API is sufficient for this project.

## 4. System Architecture

The pipeline has two phases. **Indexing** runs once per video: the transcript is fetched, chunked, embedded and stored in FAISS. **Question answering** runs for every message: the question is used to retrieve the four most relevant chunks, which are sent to Gemini together with the recent conversation.

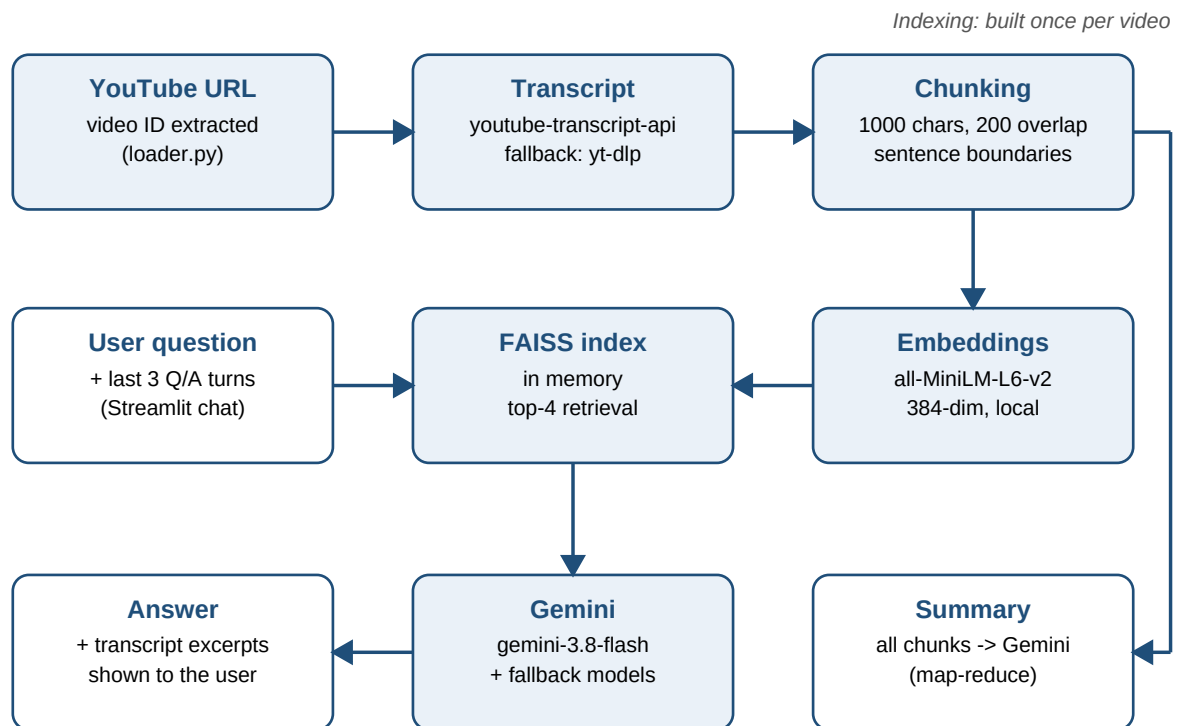


Figure 1: System architecture - indexing (top) and question answering / summarization (bottom)

### 4.1 Component summary

| Component          | File      | Description                                                                                                                                                                                |
|--------------------|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Video ID parser    | loader.py | Extracts the 11-character video ID from watch,youtu.be, shorts, embed and live links, or accepts a raw ID; rejects non-YouTube links.                                                      |
| Transcript fetcher | loader.py | youtube-transcript-api (manual captions preferred, then auto-generated; non-English transcripts translated to English when possible). Falls back to yt-dlp caption files (json3 / WebVTT). |
| Text splitter      | index.py  | RecursiveCharacterTextSplitter: 1000-character chunks with 200-character overlap, split at sentence boundaries.                                                                            |
| Embeddings         | index.py  | all-MiniLM-L6-v2 (384 dimensions, normalized), loaded once and reused.                                                                                                                     |
| Vector store       | index.py  | In-memory FAISS index; retriever returns the top 4 chunks.                                                                                                                                 |
| RAG chain          | chain.py  | Builds the prompt from the retrieved chunks, the last 3 question/answer pairs and the question; calls Gemini with automatic model fallback.                                                |
| Summarizer         | chain.py  | Map-reduce summary over all chunks, so the whole video is covered.                                                                                                                         |
| User interface     | app.py    | Streamlit chat: build knowledge base, ask questions, view transcript and the excerpts used, summarize, clear chat.                                                                         |

## 5. Methodology

### 5.1 Transcript extraction

The transcript is requested through youtube-transcript-api. For each preferred language (English first, then Hindi) a manually created transcript is preferred over an auto-generated one. Because the embedding model is English-only, a non-English transcript is translated to English when YouTube allows it. If the API fails, yt-dlp is used to locate the caption files, which are downloaded and parsed (json3 or WebVTT, with duplicate auto-caption lines removed). If both methods fail, the user sees a clear message listing why each one failed.

### 5.2 Text chunking

The transcript is split into chunks of up to 1000 characters with a 200-character overlap, so that an idea spanning a chunk boundary is still retrievable. Splits happen at sentence endings where possible, and the sentence punctuation stays with the sentence it ends.

### 5.3 Embedding and indexing

Each chunk is encoded into a normalized 384-dimensional vector with all-MiniLM-L6-v2 and stored in an in-memory FAISS index. The embedding model is loaded once per application run, and the finished knowledge base is cached per video, so returning to a video does not rebuild it.

### 5.4 Retrieval and generation

For every question, the four most similar chunks are retrieved. A follow-up such as "explain that more" carries little meaning on its own, so the previous question is added to the search query. The prompt instructs Gemini to answer only from the excerpts and to say it could not find the answer otherwise; the last three question/answer pairs are included so that follow-up questions are understood. Failed turns are excluded from this history.

### 5.5 Whole-video summarization

A summary built from only the top-4 chunks would miss most of a long video. The summarizer therefore uses map-reduce: all chunks are grouped into batches of about 12,000 characters, each batch is summarized, and the partial summaries are combined into a final 4-6 bullet summary with a one-sentence takeaway. Short videos fit in a single batch and need only one call.

### 5.6 Model fallback

Gemini models are sometimes overloaded (HTTP 503) or retired (HTTP 404). Each request goes first to the main model (gemini-3.8-flash); on any error or after a 45-second timeout it is retried on gemini-3.5-flash-lite and then gemini-3.1-flash-lite. The models are configurable in the .env file.

## 6. Implementation

### 6.1 Tools and libraries

| Category       | Library / tool                                  | Version               |
|----------------|-------------------------------------------------|-----------------------|
| Language       | Python                                          | 3.11                  |
| User interface | Streamlit                                       | 1.64.0                |
| RAG framework  | LangChain (core / community / text-splitters)   | 1.6.5 / 0.4.2 / 1.1.2 |
| LLM            | Google Gemini via langchain-google-genai        | 4.4.0                 |
| Embeddings     | sentence-transformers via langchain-huggingface | 6.1.0 / 1.2.2         |
| Vector store   | faiss-cpu                                       | 1.15.1                |
| Transcripts    | youtube-transcript-api, yt-dlp                  | 1.2.4, 2026.8.19      |
| Configuration  | python-dotenv                                   | 1.2.3                 |

### 6.2 Directory structure

```
rag_youtube_chatbot/  
|-- app.py           Streamlit chat interface  
|-- chain.py         Gemini setup, RAG chain, summarizer  
|-- index.py         chunking, embeddings, FAISS index  
|-- loader.py        video ID parsing, transcript fetching  
|-- requirements.txt  pinned dependencies  
|-- .env.example     settings template (API key, models)  
|-- rag_chatbot.pdf   this report  
|-- rag_chatbot_original.pdf original report  
`-- assets/          screenshots
```

### 6.3 Configuration

All settings live in a .env file that is excluded from version control:

| Setting               | Default                                         | Purpose                                     |
|-----------------------|-------------------------------------------------|---------------------------------------------|
| GOOGLE_API_KEY        | (required)                                      | Gemini API key (free from Google AI Studio) |
| LLM_MODEL             | gemini-3.8-flash                                | Main Gemini model                           |
| GEMINI_FALLBACK_MODEL | gemini-3.5-flash-lite,<br>gemini-3.1-flash-lite | Tried in order if the main model fails      |

### 6.4 Running the application

```
python3.11 -m venv .venv && source .venv/bin/activate  
pip install -r requirements.txt  
cp .env.example .env           # paste the Gemini API key  
streamlit run app.py           # open http://localhost:8501
```

## 7. Results and Validation

All results were produced on a MacBook (Apple silicon) with the video "What is Retrieval-Augmented Generation (RAG)?" by IBM Technology (video ID T-D1OfcDW1M): 92 caption segments, 1,062 words, 8 chunks.

### 7.1 Application screenshots

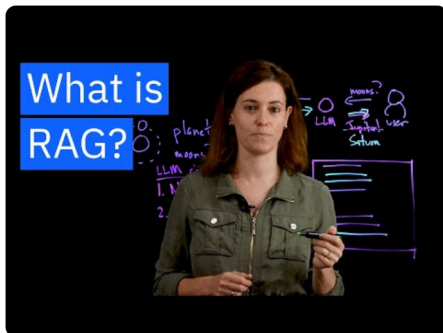
## RAG-Based YouTube Chatbot

Built using LangChain · FAISS · Google Gemini (gemini-3.8-flash)

Enter YouTube URL

<https://www.youtube.com/watch?v=T-D1OfcDW1M>

Build Knowledge Base



Knowledge Base ready

Video ID: [T-D1OfcDW1M](#)

Words: 1,062

Chunks indexed: 8

> View transcript

### Ask about the video

Summarize Video

Clear chat

RAG-Based YouTube Chatbot · Powered by LangChain, FAISS, HuggingFace & Google Gemini

Originally developed by Sohan Ghosh | MSc Data Science & AI

Figure 2: Knowledge base built for the video (thumbnail, word count and number of chunks)



What example does the speaker use about planets?



The speaker uses the question "In our solar system, what planet has the most moons?" as an example. She contrasts an outdated or incorrect answer (Jupiter) with the accurate, grounded answer (Saturn) that can be provided when using Retrieval-Augmented Generation.

▼ Transcript excerpts used

1. Large language models. They are everywhere. They get some things amazingly right and other things very interestingly wrong. My name is Marina Danilevsky. I am a Senior Research Scientist here at IBM Research. And I want to tell you about a framework to help large language models be more accurate and more up to date: Retrieval-Augmented Generation, or RAG. Let's just talk about the "Generation" part for a minute. So forget the "Retrieval-Augmented". So the generation, this refers to large language models, or LLMs, that generate text in response to a user query, referred to as a prompt. These models can have some undesirable behavior. I want to tell you an anecdote to illustrate this. So my kids, they recently asked me this question: "In our solar system, what planet has the most moons?" And my response was, "Oh, that's really great that you're asking this question. I loved space when I was your age." Of course, that was like 30 years ago.
2. And the second problem is that I am out of date. And these, in fact, are two behaviors that are often observed as problematic when interacting with large language models. They're LLM challenges. Now, what would have happened if I'd taken a beat and first gone and

Figure 3: A question answered from the transcript, with the retrieved excerpts expanded



Summarize this video.



Here is a summary of the video:

- Large Language Models (LLMs) often struggle with two major issues: providing outdated information and failing to cite credible sources for their answers.
- These models are prone to hallucinating or confidently asserting incorrect information because they rely solely on data they were trained on rather than current, verifiable facts.
- Retrieval-Augmented Generation (RAG) is a framework designed to solve these problems by requiring the LLM to search a reliable content store before generating a response.
- In the RAG process, the model combines the user's prompt with retrieved, up-to-date data, allowing it to provide grounded answers supported by evidence.
- RAG helps models be more accurate, enables them to state "I don't know" when information is unavailable, and removes the need to constantly retrain the model when new information arises.
- Current research, such as that at IBM, is focused on improving both the retrieval process and the generation capabilities to ensure the highest quality, most reliable responses.

**Takeaway:** Retrieval-Augmented Generation (RAG) improves LLM accuracy and reliability by grounding generated responses in external, up-to-date, and verified source data.

Figure 4: Whole-video summary generated with map-reduce

## 7.2 Question-answering check

Five questions were asked, four with answers stated in the video and one deliberately unanswerable. All five were answered correctly, and the unanswerable question was correctly declined instead of invented.

| # | Question                                                           | Expected                        | System answer (abridged)                                              | Result  |
|---|--------------------------------------------------------------------|---------------------------------|-----------------------------------------------------------------------|---------|
| 1 | What planet did the speaker first say has the most moons?          | Jupiter, 88                     | "The speaker first stated that Jupiter has 88 moons."                 | Correct |
| 2 | Which planet has the most moons according to the corrected answer? | Saturn, 146                     | "...the corrected answer is Saturn, which has 146 moons."             | Correct |
| 3 | What are the two LLM challenges described?                         | No source; out of date          | Out-of-date information; no source / hallucination                    | Correct |
| 4 | Who is the speaker and where does she work?                        | Marina Danilevsky, IBM Research | "...Marina Danilevsky ... Senior Research Scientist at IBM Research." | Correct |
| 5 | What recipe does the speaker recommend for chocolate cake?         | Not in the video                | "I could not find the answer in the video."                           | Correct |

A follow-up question asking what the wrong answer in *that* example was, and why it was wrong, was also answered correctly, confirming that the conversation history resolves references to earlier turns. This is a small check on one video, not a benchmark; a larger evaluation set would be needed to estimate accuracy in general.

## 7.3 Functional tests

| Feature          | Test                                                                                                | Result                                                                        |
|------------------|-----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Video ID parsing | 9 link formats (watch,youtu.be, shorts, embed, live, music, m., no scheme, raw ID); 4 invalid links | 9/9 parsed, 4/4 rejected                                                      |
| Transcript fetch | English video via youtube-transcript-api                                                            | 92 segments in 1.3 s                                                          |
| Translation      | Simulated Hindi, Bengali, blocked translation and untranslatable transcripts                        | Translated to English; falls back to the original when translation fails      |
| Caption parsers  | WebVTT (inline tags, repeated lines, hour timestamps) and json3                                     | Correct text and timestamps                                                   |
| Chunking         | Chunk boundaries of the test transcript                                                             | 8 chunks, all starting and ending on whole sentences                          |
| Chat memory      | Follow-up question referring to "that example"                                                      | Resolved correctly                                                            |
| Error handling   | Invalid Gemini API key                                                                              | Error shown in the chat; app keeps running; failed turn excluded from history |
| Display          | Text containing dollar amounts                                                                      | Shown literally, not as math                                                  |
| Model fallback   | Main model overloaded (HTTP 503)                                                                    | Answer served by a fallback model                                             |

## 7.4 Performance

| Step                            | Measured time | Notes                                                        |
|---------------------------------|---------------|--------------------------------------------------------------|
| Load embedding model            | 7.7 s         | Once per application start (plus a one-time ~90 MB download) |
| Fetch transcript                | 1.3 s         | Depends on YouTube                                           |
| Build FAISS index (8 chunks)    | 0.26 s        | Embedding + indexing                                         |
| Retrieve top-4 chunks           | 0.54 s        | Includes embedding the question                              |
| Answer a question (5 questions) | 9 - 52 s      | Dominated by Gemini latency and fallback retries             |
| Whole-video summary             | 7 - 29 s      | One call for this short video                                |

During testing the main model (gemini-3.8-flash) was usually overloaded, and most answers were served by gemini-3.5-flash-lite or gemini-3.1-flash-lite through the fallback. Answer quality with the fallback models was good on every test, but answer time varies with Gemini's load.

## 8. Changes from the Original Version

The original repository did not run as published. The table lists the problems found and how they were fixed, followed by the features added.

| Problem in the original                                                                                                 | Fix                                                                                               |
|-------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| youtube-transcript-api, yt-dlp and requests were used but missing from requirements.txt, so the app failed at start-up. | Complete, pinned requirements.txt, tested on a fresh virtual environment.                         |
| Transcript code used the removed static API (list_transcripts) and treated results as dictionaries.                     | Rewritten for the current instance API (YouTubeTranscriptApi().list(...)).                        |
| The WebVTT fallback parser always crashed (a timestamp split into four parts was unpacked into three).                  | New WebVTT and json3 parsers; yt-dlp used as a Python library; each caption format tried in turn. |
| Documentation and report described FAISS, but the code used Chroma.                                                     | Implemented with FAISS as documented.                                                             |
| Gemini model gemini-2.0-flash (and later 2.5-flash) no longer available.                                                | Current model (gemini-3.8-flash) with automatic fallback models.                                  |
| "Summarize" used only the 3 retrieved chunks, missing most of the video.                                                | Map-reduce summary over the whole transcript.                                                     |
| Each question was answered in isolation.                                                                                | Last 3 question/answer pairs sent for follow-up questions.                                        |
| When no video ID was found, the raw input was passed on as an ID, causing confusing errors later.                       | URL parser that validates the ID and reports invalid links immediately.                           |
| Dollar amounts rendered as LaTeX math; chunks began with stray punctuation.                                             | Escaped output; sentence-aware chunk boundaries.                                                  |

### Features added:

- Chat interface with history, a "Clear chat" button and the transcript excerpts behind every answer.
- Transcript viewer, video thumbnail, word and chunk counts.
- Per-video caching of the knowledge base; the embedding model is loaded only once.
- Translation of non-English transcripts to English.

- Settings in .env with a documented template; the API key is excluded from version control.

## 9. Challenges and Solutions

| Challenge                     | Cause                                         | Solution                                    |
|-------------------------------|-----------------------------------------------|---------------------------------------------|
| Application would not start   | Missing and outdated dependencies             | Pinned, tested requirements                 |
| Transcripts not retrieved     | Breaking change in youtube-transcript-api 1.x | Updated API usage plus yt-dlp fallback      |
| Gemini HTTP 404               | Model retired for new users                   | Current model, configurable in .env         |
| Gemini HTTP 503 and timeouts  | High demand on the main model                 | Fast failover to fallback models            |
| Poor retrieval for follow-ups | Short, context-dependent questions            | Previous question added to the search query |

## 10. Limitations

- Works only for videos with captions (manual or auto-generated); speech is not transcribed.
- YouTube may block transcript requests from cloud servers or VPNs.
- The knowledge base is kept in memory and rebuilt after an application restart.
- The embedding model is English-only; non-English videos depend on YouTube's translation.
- One video at a time; answer latency depends on the Gemini API's current load.
- Accuracy was checked on a small set of questions only.

## 11. Future Enhancements

- Speech-to-text (e.g. Whisper) for videos without captions.
- Timestamps in answers, linking each excerpt to its position in the video.
- Multi-video and playlist knowledge bases, with a persistent vector store.
- A multilingual embedding model for non-English videos.
- A larger, labelled evaluation set to measure retrieval and answer accuracy.

## 12. Conclusion

The project delivers a working, end-to-end RAG pipeline over real YouTube content. Transcripts are fetched reliably, indexed locally with FAISS and MiniLM embeddings, and Gemini produces answers that are grounded in the retrieved excerpts, which the user can inspect. The updated version runs on current libraries, handles failures gracefully, supports follow-up questions and whole-video summaries, and answered every question in the validation check correctly, including declining a question the video does not answer.

## 13. References

1. Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.
2. Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP 2019.

3. Johnson, J., Douze, M. and Jegou, H. (2019). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data.
4. LangChain documentation - <https://docs.langchain.com>
5. FAISS - <https://faiss.ai>
6. Sentence-Transformers - <https://www.sbert.net>
7. Google Gemini API - <https://ai.google.dev>
8. Streamlit - <https://streamlit.io>
9. youtube-transcript-api - <https://pypi.org/project/youtube-transcript-api>
10. yt-dlp - <https://github.com/yt-dlp/yt-dlp>
11. IBM Technology, "What is Retrieval-Augmented Generation (RAG)?" - <https://www.youtube.com/watch?v=T-D1OfcDW1M>